

Instruções

1. Esta avaliação deve ser feita em dupla ou em trio.
 2. Data de entrega: **07/07/2026** até as 18:55. Não pode entregar em atraso.
 3. Esta avaliação tem por objetivo consolidar o aprendizado sobre conceitos de sistemas de arquivos em sistemas operacionais. Ao concluir esta atividade, você deverá ser capaz de:
 - a. compreender a organização hierárquica de sistemas de arquivos;
 - b. compreender o conceito de inode;
 - c. compreender a diferença entre metadados e dados;
 - d. implementar gerenciamento de blocos;
 - e. implementar gerenciamento de inodes;
 - f. implementar resolução de caminhos;
 - g. implementar diretórios hierárquicos;
 - h. h. integrar um sistema de arquivos hierárquico ao microkernel desenvolvido durante a disciplina.
 4. A implementação deverá ser desenvolvida utilizando a linguagem C devido a natureza do código fonte de referência.
 5. O sistema deve ser entregue funcionando corretamente. Sistemas não compilando e executando não serão aceitos. É de responsabilidade do(s) aluno(s) apresentar a execução funcionando corretamente.
 6. Deve ser disponibilizado os códigos da implementação em repositório do(s) aluno(s) (github, bitbucket, etc...), deve ser fornecido o link e o repositório deve ser público.
 7. O relatório, em formato PDF (não é aceito outro formato), deve seguir o formato de artigo científico ou seguindo as normas da ABNT e as orientações para produção de trabalhos acadêmicos da Univali contendo:
 - Identificação do autor e do trabalho.
 - Enunciado dos projetos.
 - Explicação e contexto da aplicação para compreensão do problema tratado pela solução.
 - Resultados obtidos com as simulações.
 - Códigos importantes da implementação.
 - Resultados obtidos com a implementação (tabelas, gráficos e etc).
 - Análise e discussão sobre os resultados finais.
 8. Deve ser disponibilizado os códigos da implementação (todos os arquivos novos e modificados) juntamente com o relatório (salvo o caso da disponibilidade em repositório aberto do aluno, que deve ser fornecido o link). O repositório deve estar aberto do momento da entrega em diante, sendo que o professor não responsabiliza caso o projeto não esteja disponível para consulta no momento da correção, sendo do(s) aluno(s) essa responsabilidade de manter disponível. Cópias entre alunos implicará em nota zero para todos os envolvidos.
 9. O trabalho deverá ser apresentado em data definida pelo professor. É de responsabilidade do(s) aluno(s) explicar os conceitos, comandos, bibliotecas usadas. É de responsabilidade do(s) aluno(s) fazer a solução funcionar e ela deverá ser baixada do local de entrega no momento da apresentação. Trabalhos não apresentados terão como nota máxima 5,0, além dos descontos aplicados no restante da avaliação da implementação. Todos os alunos poderão apresentar ou o professor poderá escolher um representante para apresentar o trabalho em nome do grupo.
-

Projeto

Nesta atividade você deverá evoluir o BasicMicrokernel para incorporar um sistema de arquivos hierárquico denominado TreeFS. O objetivo é implementar um sistema inspirado nos conceitos encontrados em sistemas Unix modernos, utilizando:

- diretórios hierárquicos;
- inodes;
- gerenciamento de blocos;
- resolução de caminhos;
- criação dinâmica de arquivos e diretórios.

Ao final da atividade o kernel deverá ser capaz de navegar em uma árvore de diretórios e manipular arquivos utilizando caminhos absolutos.

Repositório de Referência

Todo o código-base utilizado nesta atividade está disponível no repositório oficial:

<https://github.com/VieF/BasicMicrokernel>

O repositório contém:

- código-fonte completo do microkernel;
- gerenciamento de memória;
- sistema de tasks;
- escalonador;
- documentação completa;
- instruções de compilação;
- exemplos de utilização.

Obs.: o aluno poderá utilizar sua própria versão desenvolvida nas avaliações anteriores ou partir da versão base disponibilizada no repositório.

Estudo Inicial Obrigatório

Antes de iniciar a implementação é obrigatório estudar:

- capítulo referente ao TreeFS;
- estrutura geral do microkernel;
- gerenciamento de memória;
- sistema de tasks;
- Makefile;
- linker.ld;
- documentação de compilação e simulação.

O objetivo é compreender como integrar o sistema de arquivos aos componentes já existentes.

Compilação e Simulação

Após clonar o repositório:

1. Instalar a toolchain RISC-V;
2. Instalar o QEMU;
3. Compilar o kernel;
4. Validar a execução da versão original antes de iniciar modificações.

Toda a implementação deverá ser executada em ambiente CodeSpace ou equivalente.

Capítulo da Documentação Utilizado

Para implementar esta atividade deverá ser utilizado como referência: Capítulo 17 – Implementação de um Sistema de Arquivos Hierárquico Baseado em Inodes (TreeFS)

Descrição da Atividade

Você deverá implementar um sistema de arquivos baseado em diretórios hierárquicos e inodes. A arquitetura mínima esperada é:

Aplicações -> API de Arquivos -> TreeFS -> Driver de Blocos -> Disco Virtual

Estruturas Obrigatórias

Sua implementação deverá possuir, no mínimo:

1. Superblock – Responsável por armazenar informações globais do sistema. Exemplos:
 - assinatura do sistema;
 - quantidade de blocos;
 - quantidade de inodes;
 - tamanho dos blocos.
2. Bitmap de Inodes – Responsável por controlar os inodes livres e ocupados.
3. Bitmap de Blocos – Responsável por controlar os blocos livres e ocupados.
4. Área de Inodes – Responsável por armazenar os metadados dos arquivos e diretórios.
5. Área de Dados – Responsável por armazenar os dados efetivos dos arquivos.

Estrutura Inicial Obrigatória

Após a formatação do sistema de arquivos, a seguinte estrutura deverá existir automaticamente:

```
/
├── home
├── tmp
└── bin
```

Essa estrutura será utilizada para validar a navegação no sistema.

Estruturas de Dados Esperadas

Inode: o inode deverá armazenar pelo menos:

- tamanho;
- tipo;
- quantidade de referências;
- lista de blocos.

Diretório: cada diretório deverá armazenar:

- nome;
- inode associado.

Arquivos Esperados

Sugere-se a criação dos seguintes módulos:

- include/treefs.h
- kernel/treefs.c
- include/block.h
- drivers/block.c
- include/inode.h
- kernel/inode.c

A organização poderá ser adaptada desde que adequadamente justificada.

Funcionalidades Obrigatórias

Sua implementação deverá fornecer:

```
int fs_init(void);
inode_t *path_lookup(const char *path);
int mkdir(const char *path);
int create(const char *path);
int unlink(const char *path);
int ls(const char *path);
int read(int fd, void *buf, uint32_t size);
int write(int fd, const void *buf, uint32_t size);
```

Gerenciamento de Inodes

Implemente um mecanismo capaz de:

- localizar inodes livres;
- reservar inodes;
- liberar inodes removidos.

Sugere-se:

```
inode_t *inode_alloc(void);
void inode_free(uint32_t inode);
```

Gerenciamento de Blocos

Implemente um mecanismo capaz de:

- localizar blocos livres;
- reservar blocos;
- liberar blocos removidos.

Sugere-se:

```
int block_alloc(void);
void block_free(uint32_t block);
```

Resolução de Caminhos

A função `path_lookup()` deverá:

1. iniciar na raiz;
2. percorrer cada componente do caminho;
3. localizar diretórios intermediários;
4. retornar o inode correspondente ao caminho solicitado.

Exemplo:

```
path_lookup("/home/aluno/notas.txt");
```

Criação de Diretórios

A função `mkdir()` deverá:

1. localizar o diretório pai;
2. verificar duplicidade;
3. alocar inode;
4. criar entrada de diretório;
5. atualizar as estruturas de controle.

Exemplo:

```
mkdir("/home/aluno");
```

Criação de Arquivos

A função `create()` deverá:

1. localizar o diretório pai;
2. verificar duplicidade;
3. alocar inode;
4. criar entrada de diretório;
5. inicializar metadados.

Exemplo:

```
create("/home/aluno/notas.txt");
```

Escrita de Arquivos

A função `write()` deverá:

1. localizar o inode;
2. calcular quantidade de blocos necessária;
3. alocar blocos livres;
4. copiar os dados;
5. atualizar o tamanho do arquivo.

Leitura de Arquivos

A função `read()` deverá:

1. localizar o inode;
2. identificar os blocos associados;
3. reconstruir os dados;
4. copiar o conteúdo para o buffer.

Remoção de Arquivos

A função `unlink()` deverá:

1. localizar o inode;
2. liberar blocos;
3. liberar inode;
4. remover entrada do diretório.

Critérios de Validação

A implementação deverá demonstrar corretamente:

- Cenário 1 – Listagem da raiz:
`ls("/")`;
- Saída esperada:
home
tmp
bin
- Cenário 2 – Criação de diretório:
 - `mkdir("/home/aluno")`;
- Cenário 3 – Criação de arquivo:
`create("/home/aluno/notas.txt")`;
- Cenário 4 – Escrita:
`write(fd, "Sistemas Operacionais", 22)`;
- Cenário 5 – Leitura:
`read(fd, buffer, 22)`;
- Cenário 6 – Remoção:
`unlink("/home/aluno/notas.txt")`;
- Cenário 7 – Navegação hierárquica:
`ls("/home")`;
- Cenário 8 – Reutilização de inodes e blocos liberados.

Funcionalidades Extras (Bônus)

Os grupos poderão implementar funcionalidades adicionais para obtenção de nota complementar:

- timestamps;
- permissões;
- links simbólicos;
- links físicos;
- blocos indiretos;
- cache de blocos;
- suporte a múltiplos usuários.

Entrega Esperada

Ao final da atividade o kernel deverá ser capaz de executar operações semelhantes às encontradas em sistemas Unix:

```
fs_init();
mkdir("/home");
mkdir("/home/aluno");
create("/home/aluno/notas.txt");
write(fd, "Bem-vindo ao TreeFS", 20);
read(fd, buffer, 20);
ls("/home/aluno");
unlink("/home/aluno/notas.txt");
```

O sistema deverá demonstrar corretamente a criação, navegação, armazenamento, leitura e remoção de arquivos utilizando a infraestrutura TreeFS implementada pelo grupo.